MANDATE — SIMULATED PROTOTYPE

Technical Documentation

A payment-intent firewall for student cross-border payments

Submission item 01 · 2026 Shenzhen International FinTech Competition, International Track, Topic C

SIMULATED — NO REAL MONEY MOVES

Team: Mandate — Arif Hussain (Team Leader), Ankon Sinha, Miftahul Alam Adib

Institution: Shahjalal University of Science and Technology (SUST), Bangladesh

Repository: github.com/arif124713/Shenzhen-International-FinTech-Competition-FinTechathon-International-Track

Live demo: mandate-nu-ten.vercel.app

Generated: 2026-10-19

Every control named here is a real `file:function` in the repository. Benchmark numbers come from `eval/reports/latest.json` (replay mode).

> Submission item 01. All results are **synthetic / sandbox**; numbers come from `eval/reports/latest.json`. Every control named here is a real `file:function` in this repository; grep the tree to verify. Wording follows the guardrails in `docs/prd.md` §0.1. If code and this document disagree, the code is authoritative and the disagreement must be logged in `docs/bugs.md` §3.

---

1. What Mandate is

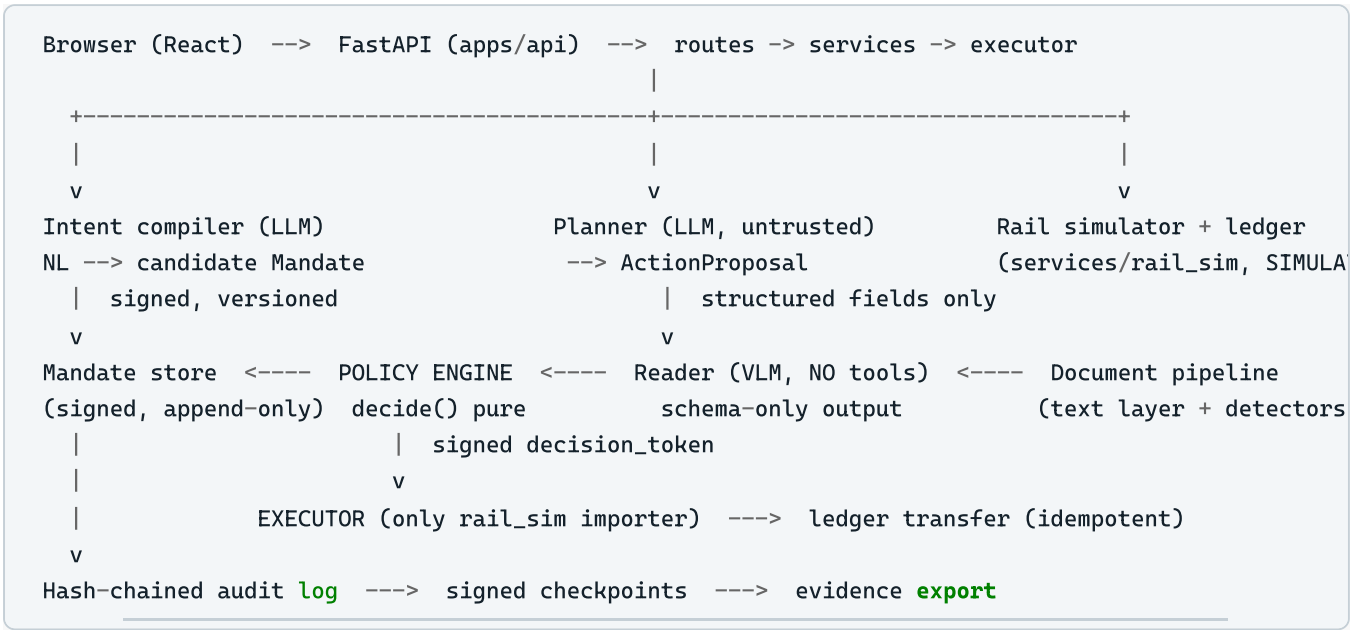
Mandate is a web app where an AI agent helps international students and their families handle cross-border money tasks (tuition, remittances, multi-currency balances), **but the agent's authority comes from a user-signed mandate, not from a prompt.** The user states rules in plain English or Bangla; the system compiles them into a typed, machine-checkable policy. An **isolated reader with no payment tools** extracts structured fields from a document. A **deterministic policy engine (plain code, no LLM)** makes every `ALLOW | HOLD | ESCALATE | DENY` decision, cross-checking the payee against a **signed registry**. Only a **signed decision token** authorizes the single execution path. Every state change is written to a **hash-chained audit log** with signed checkpoints.

One-sentence innovation. Mandate separates intelligence from authority: an AI agent may read and propose, but a deterministic engine, a signed payee registry and a transaction-bound approval decide what executes.

What it is not. This is a prototype on synthetic data. It does not move real money, does not integrate real banks or KYC, and makes no regulatory-compliance claim.

---

2. Architecture and trust boundaries



2.1 Trust boundaries

Zone	Contents	Why
Untrusted	uploaded documents, user free text, planner output, reader output *until validated*	attacker-controlled or manipulable
Semi-trusted	the LLMs (reader, compiler, planner)	can be manipulated by untrusted input
Trusted	policy engine, signature-verified registry, signed mandate, sandbox ledger, audit-log writer	deterministic code + signature checks

Boundary rules (enforced, not promised):

1. The **reader** receives the document evidence and one injected `complete()` callable and nothing else. It cannot import or receive DB, ledger, registry-write, router or executor handles. Enforced by an AST import test and a runtime signature test (`apps/api/agents/reader/__init__.py:read_document`).
2. The **planner never receives raw document text**; only sanitized `ReaderOutput` fields, the user request, a mandate summary and the ranked quote. A test double asserts this.
3. **Only** `apps/api/services/executor.py` imports `rail_sim` . An AST test (`tests/test_single_execution_path.py`) fails the build if any other module imports it.
4. The **engine** is a pure function with no network, clock or randomness; imports only stdlib + pydantic (+ `cryptography` for signature verification). Purity is enforced by a test.

---

3. Core algorithms

3.1 Policy engine (`packages/policy_engine/decide.py:decide()`)

Evaluation order (also the order shown in the Decision Trace):

1. **Integrity** — mandate signature valid, version active, proposal schema valid → else `DENY` .
2. **Anomalies** — union of reader flags + deterministic detector flags; any in `block_anomalies` → `DENY` . The security floor (HIDDEN_TEXT, INSTRUCTION_LIKE_TEXT, PAYEE_CHANGE_NOTICE) is always `DENY` even if the mandate omits it.
3. **Confidence** — `extraction_confidence_bp < min_confidence` → `ESCALATE LOW_CONFIDENCE` .
4. **Category** — not in `category_allow` → `DENY` .
5. **Registry check** — for registry-required categories, payee name + account + bank identifier must match a **signature-valid** entry exactly after NFKC normalization and confusable detection. Mismatch → `DENY REGISTRY_MISMATCH` ; bad signature → `DENY REGISTRY_SIGNATURE_INVALID` .
6. **Payee status** — allowlisted or valid-registry payee is *known*; else *new payee* → `new_payee` action (ESCALATE) and `cooling_off` (HOLD).
7. **Caps** — per-tx cap, then rolling-window caps computed from the **ledger** (not the proposal).
8. **Escalation thresholds** — `escalate_above` → `ESCALATE` .
9. **Rail / time-window rules** → `DENY` if violated.
10. **Currency sanity** — currency in the allowed set; `0 < amount ≤ balance` else `DENY` .

Precedence: `DENY` > `HOLD` > `ESCALATE` > `ALLOW` ; strictest wins; **default deny**. Rules are sorted by `id` inside `decide()` so rule order cannot change an outcome.

Signed decision token (`tokens.py:issue_decision_token`). The engine returns a signed token binding `{decision, tier, action_hash, mandate_id, mandate_version, registry_version, input_hash, issued_at, expires_at, nonce}` , where `action_hash` covers `{action, category, payee_id, payee_ref, amount_minor, currency, source_account, source_debit_minor, rail_id}` . A changed fee or debit therefore cannot ride on an earlier ALLOW.

3.2 Single execution path (`apps/api/services/executor.py:execute()`)

The executor accepts only `(proposal, decision_token)` and refuses unless **all** hold: signature valid and key active; `decision == ALLOW` ; unexpired; nonce unused; `action_hash` recomputed from the proposal being executed equals the token's; the mandate version is still active; the registry entry is still valid. Any failure → no execution and an `EXEC_REFUSED` audit entry with a reason code. The nonce check, ledger write and nonce burn happen in one critical section (thread-safe).

3.2b Duplicate payment — decision-layer idempotency (`policy_engine/decide.py` , `Context.paid_action_hashes`)

Paying the same invoice twice is a direct financial harm, so it is prevented at the **decision** layer, not only at execution. `action_hash` binds payee + amount + currency + debit + rail and **excludes** `proposal_id` , so a re-submitted invoice (a new proposal object for the same underlying payment) hashes identically. The ContextBuilder populates `Context.paid_action_hashes` from the sandbox ledger's executed receipts; if the proposal's `action_hash` is in that set the engine returns `DENY DUPLICATE_PAYMENT` before any other rule.

This is distinct from, and complements, two existing guards: the executor's `TOKEN_REPLAY` (stops the *same token* being reused) and the ledger's idempotency key (stops the *same key* debiting twice). Duplicate detection catches a **new request** for an already-paid invoice. It is strictly additive — a new DENY reason — so it never relaxes another rule.

Scope and honesty: the match is on the canonical action (payee identifiers, amount, currency, debit, rail). It does **not** attempt fuzzy "same invoice with OCR variation / different formatting" matching, which would require the issuer, invoice reference and date and is a correctness risk; that remains a documented limitation.

3.3 Approvals are inputs, not bypasses (`apps/api/services/approvals.py`)

A 256-bit random approval token is stored **hashed only**, single-use, expiring, and bound to `action_hash + mandate_version + approver + expiry` . On approve/reject the **engine is re-run** with the approval in its context; a new decision token is issued only if the engine now returns `ALLOW` . Rejection is final for that action hash. Comparison is constant-time (`hmac.compare_digest`).

3.4 Routing (`apps/api/services/router.py`)

For a corridor `S>D` and amount `A` : candidate rails are those supporting the corridor and allowed by `rail_allow` . Cost = `fixed_fee + pct_fee·A + intermediary_fee + receiving_bank_fee` , all in `Decimal` , rounded half-even to minor units. Rails that cannot meet the deadline are excluded with a reason. Ranking: `normal` → lowest cost then speed; `urgent` → fastest then cost. **Savings are versus the simulated default rail, never an arbitrary expensive baseline.** Rate cards carry `source_url` , `source_date` , `assumptions` , `limitations` ; a missing field fails the build.

3.5 Confidence derivation (`apps/api/agents/reader/schema.py`)

The model's self-reported confidence is weak evidence. The final `extraction_confidence_bp` starts from the model value and is capped or reduced by agreement signals: whether amount/account/payee also appear verbatim in the text layer, whether text-layer parsing and the model agree, and the image-quality measure. Components are recorded so the audit view can show *why* confidence was low. Values are integer basis points (canonical JSON forbids floats).

3.6 Registry check and trust (`apps/api/services/registry.py`, `policy_engine/decide.py`)

Payee resolution happens in the planner, not the engine. The planner is the only component that resolves a document-derived payee to a registry entity: `apps/api/agents/planner/_resolve_payee_id` looks the payee up in the read-only registry by its identifiers (NFKC-normalized name + account + bank) and sets `payee_id`. The engine then requires a `payee_id` and verifies that the referenced entry matches the proposal's identifiers — so a fabricated `payee_ref` cannot claim an entry without that lookup (`payee_id is None` ⇒ `DENY`).

An entry is valid only if its signature verifies with a **non-retired** key, `valid_from ≤ now ≤ valid_to`, and it is not revoked. **A signature proves a record was not altered; it does not prove the institution is genuine.** Provenance fields (`method`, `source_url`, `retrieved_on`, `second_check`) are required and shown in the UI trust badge, with the sentence "A signature shows the record hasn't been changed. It doesn't prove the institution is genuine."

Registry UI: `GET /api/v1/registry` lists signed entries; `POST /api/v1/registry` adds a team-enrolled, signed, audited entry (provenance captured; identifiers canonicalized as the reader sanitizer does); `POST /api/v1/registry/{entity_id}/revoke` marks an entry revoked and re-signs it.

3.7 Audit log and checkpoints (`apps/api/services/audit_log.py`)

Each entry stores `hash = sha256(prev_hash || canonical_json(entry))`. `verify()` recomputes the chain and returns the first bad sequence number. The head hash is periodically signed with a **separate checkpoint key** and exported, so a full-log rewrite is detectable against an externally held checkpoint. Residual risk: if both the DB and the checkpoint key are compromised, the log can be forged (R-022).

---

4. Security design (summary — full detail in `docs/security_self_assessment.md`)

Control	Enforcement point
Deterministic decision + signing	`policy_engine/decide.py:decide()`, `tokens.py:issue_decision_token()`
Execution verification	`apps/api/services/executor.py:execute()`
Single execution path	AST test `tests/test_single_execution_path.py`
Reader isolation	`apps/api/agents/reader/__init__.py:read_document`
Registry check	`policy_engine/decide.py` (registry step), `apps/api/services/registry.py:entry_valid`
Approval binding + single use	`apps/api/services/approvals.py`
Audit chain + checkpoints	`apps/api/services/audit_log.py:append()/checkpoint()`
Engine purity	AST import scan `packages/policy_engine/tests/test_engine_purity.py`
Upload validation	`apps/api/services/documents.py:validate_upload`

Upload rules: PDF/PNG/JPG decided by **content sniffing** (not extension or client MIME), ≤ 10 MB, zero-byte / encrypted / >5-page PDFs rejected, random server-side filenames stored outside any static root, SHA-256 computed, embedded scripts never executed, metadata stripped from anything shown.

Approval endpoint: accepts only the token path parameter plus approve/reject; exposes only the pre-payment summary fields. **Demo-only mutating routes** (`/admin/reset`, `/clock/advance`) are disabled unless `DEMO_MODE=1`.

---

5. Data model

SQLAlchemy 2 models live in `apps/api/db/models.py` (matching `implementation_plan.md` §5.1): `users`, `mandates`, `payees`, `documents`, `proposals`, `approvals`, `accounts`, `ledger`, `audit_log`. Datetimes are UTC. The demo runtime (`apps/api/services/runtime.py`) keeps an in-memory sandbox ledger and audit log so the deterministic demo and the offline benchmark run without a database; SQLite is the persistence target for a durable run.

Money is **integer minor units** everywhere (`amount_minor: int`); FX rates and percentages are `Decimal`. Rounding is half-even to minor units; `float` is never used for money.

---

6. LLM integration

All LLM calls go through the `LLMClient` interface (`apps/api/agents/llm.py`): an OpenAI-compatible HTTP client (for DeepSeek via its Chat Completions endpoint), a `ReplayClient` (offline fixtures keyed by `(prompt_version, canonical messages)`), and a `FakeClient` for unit tests. Base URL, API key and model

IDs come from environment variables; nothing is hard-coded. Temperature 0; reasoning/thinking off for extraction and compilation. Invalid or empty output → one retry → **fail closed** (ESCALATE with a typed code, never a best-effort parse). Only synthetic documents ever leave the machine.

Record/replay: every LLM-dependent test and every benchmark run uses `ReplayClient` fixtures (`data/fixtures/llm_replay/`); `make test` and `make eval` need no network.

---

7. Evaluation

`make eval` runs, offline and deterministically (`EVAL_SEED=1337`):

- **Red-team benchmark** (`eval/run_eval.py`): 96 attack cases (24 families × 4) plus 4 benign look-alikes, against **B0** (no-policy baseline), **B1** (prompt-only baseline) and **B2** (Mandate). An attack succeeds only if a **violating ledger action** occurs — measured from the ledger, not from the agent's words. **Important:** this is an **engine-level** benchmark — each case supplies a labelled `ReaderOutput` and anomaly flags, so the result measures the deterministic engine *given correct fields*. B0/B1 are structural baselines (deterministic doubles in replay mode), not hosted-model agents; it is **not** a live-LLM end-to-end evaluation.
- **Reader/detector evaluation** (`eval/reader_eval.py`): the front of the pipeline (real upload → detectors → reader) on the labelled invoice set — the layer the engine benchmark does not cover.
- **Task-completion scenarios** (`eval/scenarios/run.py`): the priority subset (T01, T02, T06, T07, T08, R02, R05, R06, R07, P01) run headless against the real engine, registry, executor, router and audit log.
- **Evidence index** (`docs/execution_evidence/index.md`): maps every scenario id and attack family to its result.

Measured on this repo (`eval/reports/latest.json` , replay mode): **engine-level** attack success **B0 69.8% · B1 40.6% · Mandate 0%** (96 attack cases; 95% Wilson CI [0, 0.039]); false-block result 0/4. Reader/detector layer: anomalies caught 2/2, genuine documents clean 3/3, field agreement 9/9. This is a **static**, synthetic, **replay** benchmark; adaptive attackers may do better (R-003), and a live-LLM run is a documented to-do (R-012), not completed evidence.

---

8. Limitations

1. Sandbox only: no real money, banks, KYC or identity checks.
2. All documents, users and balances are synthetic; results may not hold on real data.
3. The benchmark is static; an adaptive attacker may succeed where our tests did not.
4. The policy compiler can still be wrong; the safety loop reduces but does not remove this risk (R-001).
5. The reader can misread documents; low-confidence cases are escalated, not guaranteed correct (R-002).
6. Rate cards are simulated; savings are relative to a simulated default rail (R-009).
7. The Bangladesh rule pack is unverified placeholder content, not legal advice (R-010).
8. Registry governance, key custody and identity are prototype-grade (R-021, R-026).
9. A wrong-but-permitted extraction (a wrong amount under the cap) can still pass; the LLM has no authority to execute, but it can influence what is proposed (R-002).

9. Reproduction

```
make setup && make test && make lint     # offline engine + API tests
make eval                                # regenerate eval/reports/latest.{json,html} + evidence i
make e2e                                 # Playwright UI flows (boots API + Vite)
```

Then open `eval/reports/latest.html`, or `GET /api/v1/eval/report`. Evidence artifacts for the submission are written by `python -m eval.evidence` (audit export, ledger CSV, signed checkpoint, LLM config note) and `make eval` (report + `docs/execution_evidence/index.md`).